NATIONAL UNIVERSITY OF SINGAPORE

Institute of Systems Science

Certificate in Intelligent Reasoning Systems

Practice Module

RoadGuard

Road Damage Detection and Maintenance Scheduling in Jurong (SG)

Final Project Report

AIS08FT · Group 37

Hu Erji A0305480L

Yan GuanTing A0305485B

Wu TaoYi A0356972M

Abstract

RoadGuard combines road-damage detection, rule-based decisions and maintenance scheduling for crews in Jurong. Inspectors upload geolocated images, and a local YOLO11s model selects one primary damage observation. A confidence policy then determines whether to create a repair task or request human review. A genetic algorithm and a Greedy baseline allocate accepted work from a common NUS-ISS dispatch origin. Inspectors, dispatchers and crews use a shared web application that retains task history and checks record revisions. A dispatcher assistant retrieves reviewed project knowledge and evidence from saved plans. Separate preview and confirmation workflows allow dispatchers to save crew assignments and cancel tasks.

Our evaluation combines visual-recognition experiments, twelve routing scenarios and tests of the integrated workflow. The GA initialized with a Greedy solution improved on Greedy in all 120 runs under the default search settings. Mean objective reductions across scenarios ranged from 0.73–3.62%. We evaluated the assistant using actual GPT-generated responses and local retrieval tests, and separately verified user-confirmed task changes. The project connects inspection evidence to crew planning while allowing human correction and retaining a record of decisions.

Keywords: road damage detection; YOLO; business rules; genetic algorithm; maintenance scheduling.

Contents

Abstract ..... **i**

1 Introduction **1**

1.1 Background Problem and Business Value 1

1.2 Related Work and Contribution 1

1.3 Scope and Project Evolution 2

2 Data and Knowledge Preparation **4**

3 System Design and Implementation **6**

3.1 System Architecture 6

3.2 Detection Decisions and Route Scheduling 6

3.3 Implementation and User Workspaces 8

3.4 Application of Intelligent Reasoning Techniques 12

3.5 Team Contributions and Development Practice 12

4 Evaluation Methodology **13**

5 Results and Discussion **15**

5.1 Vision Performance and Error Analysis 15

5.2 Routing Performance 18

5.3 Integrated Workflow Verification 20

6 Conclusions and Future Work **23**

6.1 Conclusions and Limitations 23

6.2 Future Work 24

Appendix A Installation and User Guide **25**

A.1 Environment and dependencies 25

A.2 Local retrieval and optional GPT configuration 25

A.3 Accounts and data 27

A.4 Start the application 27

A.5 User workflow 28

A.6 Dispatcher evidence questions 29

A.7 Dispatcher route previews and confirmed adjustments 30

A.8 API testing and troubleshooting 31

A.9 Optional Google travel estimates 32

Appendix B Submitted Project Proposal **34**

Appendix C Course Functionality Mapping **47**

1 Introduction

1.1 Background Problem and Business Value

RoadGuard addresses the handoff between road inspection and maintenance scheduling. A photographic observation must be interpreted, reviewed when uncertain, assigned a repair priority and allocated to an available crew. Detection alone does not establish road eligibility, service time or operational urgency.

Singapore's Land Transport Authority describes manual inspection and reporting as labour-intensive and dependent on inspectors' judgement. Its published account of AI-assisted inspection and GeoFormSG also identifies the effort required to transfer location coordinates between applications. These observations highlight a practical challenge in Singapore's existing road inspection process. We developed RoadGuard as an independent prototype.(System is not integrated with LTA systems)

RoadGuard aims to reduce repeated data handling by connecting inspectors, dispatchers and crews through one traceable workflow. The same inspection record supports review, repair-task allocation and completion, reducing the need to transfer information between separate tools. Explicit repair rules and comparisons of crew plans give dispatchers a consistent basis for prioritising work and checking available capacity. Crews can view their assigned routes and record completion, while saved evidence and task history help explain decisions and follow up outstanding work. These capabilities are intended to improve coordination and accountability; any savings in staff time or operating costs require evaluation in actual use.

1.2 Related Work and Contribution

Arya et al.'s RDD2022 benchmark contains 47,420 images from six countries and covers four damage families. RoadGuard uses the separately published eight-class RDD2022ES derivative. Redmon et al.'s YOLO work provides the methodological basis for predicting bounding boxes and classes jointly. Our implemented detector uses the later Ultralytics YOLO11, which we selected through project experiments.

For scheduling, Vidal's research on hybrid genetic search illustrates the value of combining evolutionary search with local improvement in capacitated routing. RoadGuard uses a smaller

custom GA rather than Vidal's HGS or SWAP* method. A deterministic Greedy baseline provides a transparent basis for comparison. Our contribution is the combination of learned perception, editable rules, human correction and evaluated crew planning, rather than a new detector architecture or search algorithm. Compared with an inspection-only tool, RoadGuard extends the workflow from accepted evidence to repair allocation and completion.

1.3 Scope and Project Evolution

Four crews serve eligible public motor roads in Jurong East and Jurong West within the frozen URA Master Plan 2025 boundaries, using a simulated NUS-ISS origin. Expressway mainlines and ramps, as well as private or restricted roads, are excluded. The backend checks whether a location falls within the permitted area, while users declare the road type.

The workflow covers primary damage detection, confidence-based review, rule-based task creation, comparative scheduling and crew completion. Accepted inspections trigger automatic planning attempts, while other changes require explicit replanning. The dispatcher assistant also provides answers grounded in evidence and supports confirmed adjustments. Table 1 compares the delivered scope with the submitted proposal in Appendix B. The restriction to Jurong and the use of four crews follow presentation feedback; the other changes reflect implementation decisions or gaps in evaluation.

Table 1. Changes from the original project plan

Item	Original plan	Current evidence
Area and crews	Singapore example with two crews	Jurong East/West, four crews, NUS-ISS origin
Wide crack priority	Wide, sunken or deep labels mapped to P1	Wide cracks use P2; sunken/deep labels retain P1
Route refresh	Queue changes automatically trigger recalculation	Accepted uploads and review confirmations or corrections trigger a planning attempt after the task is committed; other changes require explicit recalculation
Travel and map	Google Routes and Maps JavaScript API	Optional Google travel matrix; simulated travel by default; schematic SVG map

Item	Original plan	Current evidence
Language assistant	GPT/RAG explanations and confirmed change requests	Chroma/E5 and optional GPT answers; confirmed adjustments; targeted evaluation
Business rules	Proposed rules.yaml and provisional mapping	Versioned JSON policy with 60/120/240-minute durations
User access	Inspector, coordinator and crew workflows	Authenticated roles; inspectors restricted to their own inspections and crew accounts to their fixed crew

2 Data and Knowledge Preparation

We used version 1 of the RDD2022ES dataset and retained its eight damage classes throughout model development. Our preparation pipeline decodes images and checks annotation coordinates and class IDs. It sets aside groups with missing labels or uncertain orientation, and records these decisions in exclusion manifests and file hashes.

We group original images with their mirrors before splitting the data, and use exact and conservative perceptual similarity checks to link other related images. We then allocate groups by country using seed 42 and approximate proportions of 70/15/15. Training includes mirrors, while validation and test use only original images. This procedure limits known duplicate leakage, but without sequence or location metadata it cannot establish that adjacent scenes are independent.

Table 2. Prepared visual-data partitions

Partition	Images	Groups	Annotated boxes	Use
Training	17,916	8,939	25,314	Originals and supplied mirrors
Validation	1,921	1,917	2,623	Originals only; model selection
Test	1,918	1,915	2,673	Originals only; historical n10 test

The research photographs come from foreign sources. The synthetic Jurong task coordinates are separate scheduling inputs and do not establish where any photograph was captured.

Versioned business rules specify severity, priority, service time in minutes and explanations. The a-knowledge-v19 package records the reviewed scope, label meanings, rules and workflows, together with hashes of its sources. We index it locally in ChromaDB using multilingual-e5-small embeddings. The package was checked against repository content. Only reviewed static documents enter the index; facts from saved plans are obtained separately with authorization checks. The visual qualifiers “wide”, “sunken” and “deep” are categorical labels rather than measurements of width, depth or structural condition.

Table 3. Damage classes and demonstration rules

Class	Severity	Priority	Service minutes
longitudinal crack	low	P3	60
longitudinal crack wide	medium	P2	120
transverse crack	low	P3	60
transverse crack wide	medium	P2	120
alligator crack	medium	P2	120
alligator crack sunken	high	P1	240
pothole	medium	P2	120
pothole deep	high	P1	240

Confidence determines whether an observation is accepted, while the selected label determines the rule outcome. An incorrect label with high confidence can therefore create an incorrect task. Service times are project assumptions rather than measured construction standards.

3 System Design and Implementation

3.1 System Architecture

The React frontend connects to one FastAPI backend and a shared SQLite database. Separate services handle vision, rules, inspections, tasks, crews and planning. YOLO predicts the damage class and its location within the image for review. Repair policy is held in explicit rules, so it can change without retraining the model. The GA searches for suitable task assignments and visit orders, while Greedy provides a baseline and an initial solution.

The dispatcher assistant retrieves project documents from a local ChromaDB index using E5 embeddings and obtains authorised facts from saved plans separately. When GPT generation is enabled, the model selects supporting evidence IDs supplied by the server. The server attaches exact quotations and checks the answer. For repair-rule questions, it constructs the answer from the matching rule rows. Changes to tasks and crews follow a separate service that validates the request and saves it only after confirmation; an answer alone cannot authorise an action.

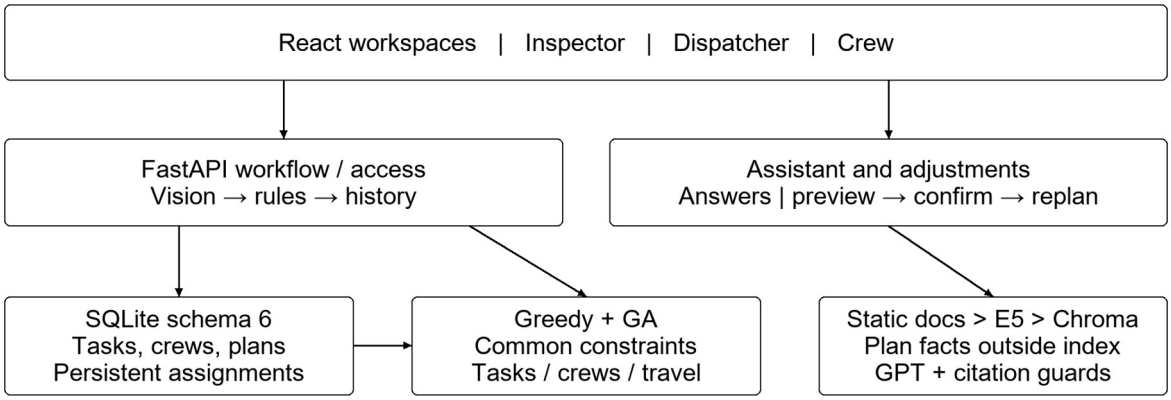


Figure 1. System architecture and principal information flows.

Note. Questions do not change business records. Dispatchers must confirm proposed adjustments and explicitly recalculate the affected routes.

3.2 Detection Decisions and Route Scheduling

Before loading the model, the backend verifies that its configuration, weight file and eight damage-class labels match the expected definitions, including the mapping between class IDs

and labels. Inference filters candidates at confidence 0.20. It ranks primary-box candidates by confidence multiplied by the fourth root of the fraction of image area they occupy; this ranking score does not replace raw confidence. A primary observation with raw confidence of at least 0.60 automatically creates one task. Lower confidence requires review, and no detection creates no task. The system retains raw evidence, the rule snapshot and review history. Corrections use the frozen rules, and idempotent requests prevent duplicate tasks.

Both planners read the same active tasks, crew availability, travel matrix and constraints at a consistent revision. The Greedy Algorithm repeatedly selects the crew that becomes free earliest, then assigns it the feasible task with the highest priority and, among equal priorities, the shortest travel time. The GA represents each solution as a task permutation and a vector assigning tasks to crews. It explores alternatives through tournament selection, order crossover (OX) for the permutation and uniform crossover for the crew assignments, mutation by swap, segment reversal, reassignment or relocation, and elitism, while caching evaluations to avoid scoring the same solution twice.

The deployed planner inserts the Greedy solution into the GA's initial population, and elitism retains the best solution in every generation, so the online GA can never return a worse plan than Greedy under the shared ranking. The experiments additionally evaluate a pure GA without this seed, so that comparisons with Greedy remain fair and the effect of seeding can be measured. Both the Greedy and GA planners respect saved task-to-crew assignments, including after a system restart or when new tasks trigger recalculation. If the assigned crew is unavailable or cannot accommodate the task within the planning constraints, the task is marked as unassigned. It is not automatically transferred to another crew.

The planners compare candidates in order of unassigned P1 tasks, then unassigned P2 tasks, then unassigned P3 tasks. Only when these counts are equal do they minimise J = makespan + 0.2 × total travel + 0.5 × workload standard deviation + 0.5 × priority-weighted mean completion time. This ordering gives task coverage precedence over the weighted objective. Makespan is the longest crew schedule, and workload standard deviation measures differences in work duration between crews. The final term averages task completion times using P1/P2/P3 weights of 3/2/1. All components use minutes, but J is a weighted score rather than elapsed time or money. Since tasks have no deadlines, priority-weighted completion does not measure lateness.

Accepted uploads and review confirmations or corrections automatically trigger an attempt to plan with both GA and Greedy after the task is committed, using the same snapshot and matrix. Pending review, no detection and dismissal do not trigger planning. Completion, cancellation, crew changes and assistant confirmations require explicit replanning. Before saving the paired plans, the system rejects any input revision that has been superseded.

Since dispatchers can move tasks between crews, the system previews each proposed move to check that the revised schedule remains feasible. It keeps all other assigned tasks in place and checks whether travel, repair work and any required return journey fit within the shift limit. Once the dispatcher confirms the move, the new crew assignment is saved and respected in future planning. Cancelling a task removes it from active work but preserves its history.

The system prevents outdated previews, conflicting changes and repeated confirmations from causing unintended updates. It also checks user permissions and protects against unauthorised requests from other websites. After reassignment, the previous crew immediately loses access to the task and its inspection record and can no longer mark it as complete. The receiving crew gains access once a new GA route plan has been saved. Before recording task completion, the system checks that the task has not changed since the crew retrieved it..

3.3 Implementation and User Workspaces

The application supports uploads, inspection details, reviews, tasks, crew availability, route planning, algorithm comparison and a mobile crew workspace. Backend authorization enforces the role restrictions in Table 4 independently of the navigation shown to users. Authentication uses hashed passwords and sessions, with CSRF checks for modifying requests.

The Route assistant presents its answer in English first, with supporting sources available in a “View sources” section that is collapsed by default. It also displays the saved routes for all four crews and lets dispatchers preview task moves or cancellations before confirming them.

When asked which model it uses, the assistant reads the model name directly from the backend settings without contacting the AI service or exposing sensitive information. For questions clearly outside the project’s scope, it explains what it can help with. Answers about repair rules and saved routes must be supported by relevant evidence and pass the system’s

citation checks. If the question is unclear or the available evidence is insufficient, the assistant states this explicitly.

Table 4. Account roles and access restrictions

Role	Permitted workflow	Server-side restriction
Inspector	Upload and inspect own results	No global tasks, reviews or planning
Dispatcher	Review inspections; manage tasks and crews; plan and compare routes; ask evidence questions; confirm adjustments	Cannot upload or impersonate a crew workspace
Crew	Read own latest GA route and complete assigned work	Cannot select another crew, replan, adjust tasks or access global lists; reassignment revokes old access

The saved routes discussed by the assistant depend on the travel estimates used during planning. By default, the planner uses a simulated matrix that converts straight-line distance, multiplied by 1.3, into travel time at 30 km/h. Google Routes estimates are used only when an administrator selects that provider and supplies a server-side API key; selecting Google without a valid key produces an explicit error rather than a simulated result. The system checks and caches them for reuse. If the selected source fails, it reports an error rather than silently switching to another source. To help dispatchers interpret the results, the interface identifies the source, flags plans that need updating and explains why some tasks remain unassigned. Routes are shown as schematic connections between stops rather than detailed driving directions. Percentage comparisons between algorithms are shown only when the planning conditions and assigned workloads are comparable.

We tested these features as part of the complete workflow, from image upload and damage detection to task creation, route planning and assistant queries. This integration testing also revealed an unexpected interaction between the vision library and the upload process. Loading the library replaced the image decoder used for uploads, causing malformed images to produce a server error instead of a normal rejection. We retained the original validated decoder and added a regression test to prevent the problem from recurring. This experience showed why testing individual components was not sufficient: we also needed to verify how they behaved when used together.

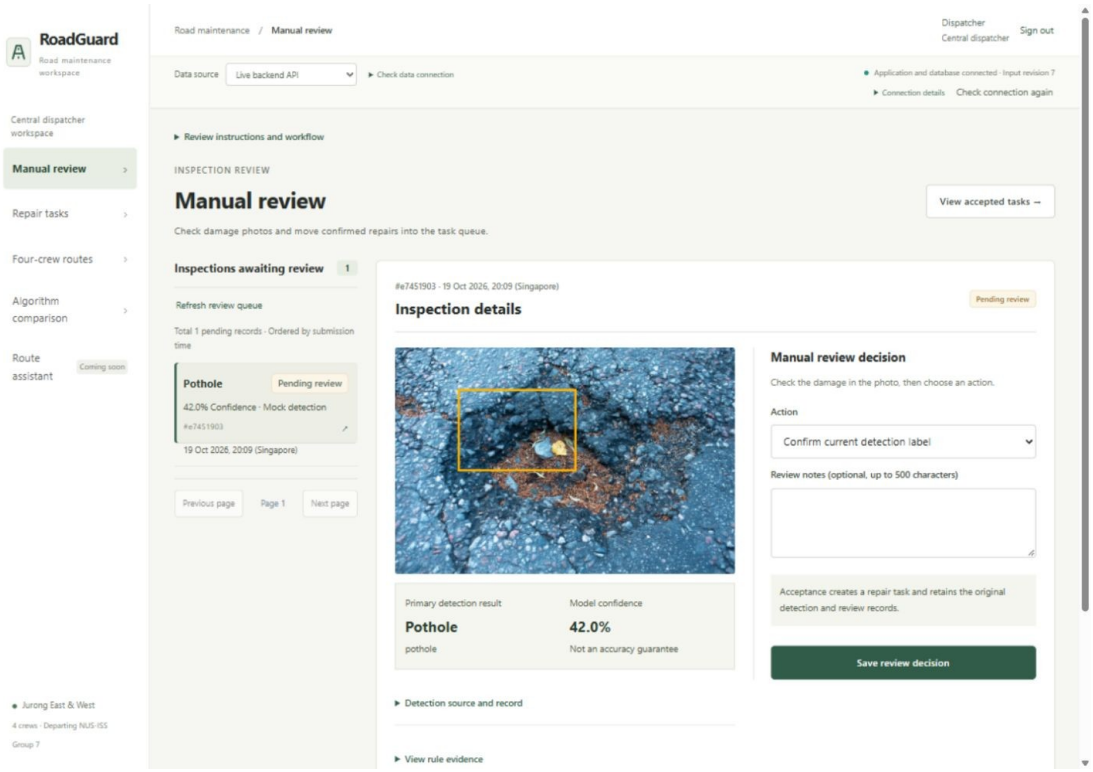


Figure 2. Dispatcher review interface showing a synthetic confidence case. Capture dated 19 October 2026.

Note. The 42% result is a controlled review case rather than a model-accuracy measurement.

On mobile, crew accounts can sign out, see their fixed crew identity, view the current route and confirm task completion. Changing the URL cannot grant access to another crew's work. Recorded browser checks at widths of 360 and 390 pixels found no horizontal overflow across the page as a whole.

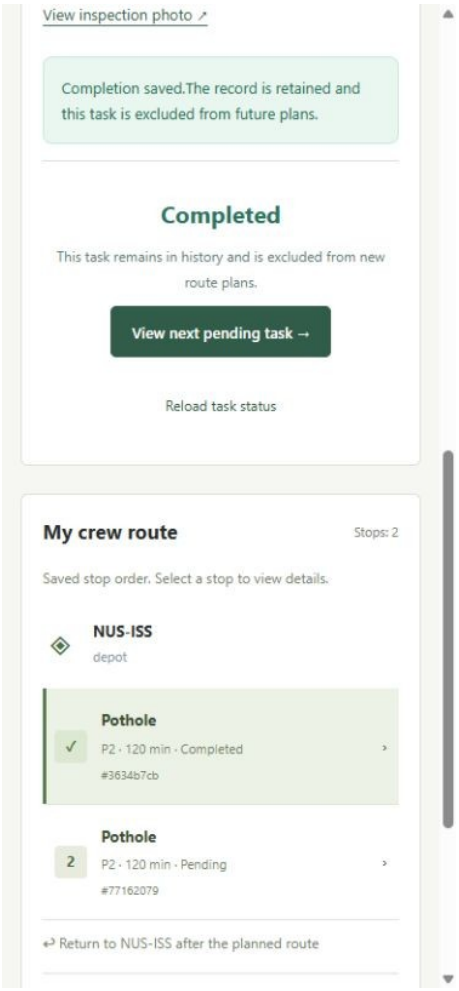


Figure 3. Mobile crew workspace showing task completion and route entries.

Note. Recorded on 19 October 2026. Completion is checked against the current crew assignment and task version.

3.4 Application of Intelligent Reasoning Techniques

The methods serve different roles in the same workflow. YOLO supplies a predicted damage type and location, with uncertainty represented by its confidence score. Explicit rules translate accepted observations into repair tasks, and constrained search assigns those tasks to crews. The assistant retrieves reviewed knowledge and saved-plan facts to explain decisions, with optional language generation and checks against the cited evidence. Sections 3.1 and 3.2 describe how these steps interact, while Appendix C maps them to MR, RS and CGS and identifies the supporting evaluation and its limits.

3.5 Team Contributions and Development Practice

Table 5. Team responsibilities and completed contributions

Member	**Primary area**	**Completed contribution**
Hu Erji A0305480L	Vision and shared workflow	Data preparation and YOLO model experiments and training; business rules and shared backend; model release; integration, authentication and later interface refinement.
Yan GuanTing A0305485B	Routing and GA algorithm optimisation	Greedy and GA planning for four crews with a shared priority-first evaluator; Greedy-seeded online GA; persistence of crew and plan records; Google matrix provider; routing experiments, ablation, best-known-solution evaluation and diagnosis of tight-capacity failures; frontend visual redesign and usability fixes.
Wu TaoYi A0356972M	Frontend and assistant	Frontend foundation and workflow pages; map and comparison integration; assistant for retrieval and confirmed changes. Shared integration added vector-based grounding and persistent task adjustments.

4 Evaluation Methodology

We evaluated recognition quality, scheduling quality and workflow correctness separately. The paired YOLO11n/YOLO11s experiments used COCO-pretrained initialisation, 30 epochs, 640-pixel inputs, a batch size of 32, AdamW and seed 42. Subsequent YOLO11s fine-tuning used 40 additional epochs, 832-pixel inputs and a batch size of 16. Because multiple settings changed together, the resulting gains cannot be attributed to a single parameter.

We measure precision as TP/(TP+FP), recall as TP/(TP+FN), and F1 as their harmonic mean. TP, FP and FN denote true positives, false positives and false negatives. For box evaluation, a correct detection must have the right one of eight class labels and match an annotation at intersection over union (IoU) of at least 0.5. Each prediction and annotation can be matched only once. We pool the counts across images to calculate precision and recall; these results differ from the class-averaged values and mean average precision (mAP) reported by the native validator.

The single-subject evaluation asks whether the selected detection has the correct primary damage type. It does not require the box to overlap the annotation. A wrong label counts as both a false positive for the predicted class and a false negative for the true class. This lets us examine classification and missed detections separately from box placement.

We split the 1,230 images with zero or one annotation deterministically by source group into 853 calibration cases and 377 confirmation cases, and considered the 691 multi-object images separately. We compared eighty policies using only the calibration subset. Both subsets came from previously used validation data, so the confirmation subset is not an independent test. Macro F1 averages all eight classes, assigning zero to undefined class ratios, including the unsupported wide-transverse class in the confirmation subset.

We evaluated the routing algorithms in 12 scenarios, each with four crews. The scenarios combined three workloads of 10, 15 or 20 tasks, two sources of travel-time estimates (simulated or Google), and two working-time settings (a 480-minute shift limit or no limit). For each scenario, we ran Greedy once and tested both GA approaches ten times using random seeds 1–10. One GA approach started without a Greedy solution, while the other included it as an initial candidate. We also performed 20 additional runs with more extensive searches, using seeds 101–

120, to establish a reference for comparison. This produced 41 runs per scenario and 492 runs overall.

In the standard experiments, the GA maintained 60 candidate solutions and searched for up to 300 generations, stopping early if the best solution did not improve for 80 consecutive generations. The crossover and mutation probabilities were 0.9 and 0.3. Parents were selected through tournaments of three candidates, and the two best solutions were carried forward to the next generation. The reference runs increased the population to 150 candidates and allowed up to 1,000 generations, with early stopping after 300 generations without improvement. These runs alternated between starting with and without a Greedy solution. Within each scenario, all runs used the same task data, travel estimates, constraints and evaluation method to ensure a fair comparison.

5 Results and Discussion

5.1 Vision Performance and Error Analysis

In the paired 30-epoch comparison, we selected YOLO11s because it achieved higher validation mAP50–95. At confidence 0.25, however, it produced 107 more correct matches and 220 more false positives than YOLO11n. We measured warm CPU latency under matched conditions on 40 images, repeating the measurement three times. Median/P95 latency was 53.27/63.59 ms for n30 and 81.87/91.85 ms for s30. These timings include image preparation and inference but exclude cold loading, HTTP, database operations and task processing.

Table 6. Validation detection results on the same 1,921 images

Model / size / confidence	mAP50	mAP50–95	P (%)	R (%)
n10 / 640 / 0.25	34.28	16.93	52.48	41.17
n30 / 640 / 0.25	37.27	19.16	50.76	47.12
s30 / 640 / 0.25	38.77	20.23	48.62	51.20
Balanced / 832 / 0.25	40.16	20.27	50.52	52.00

Note. The model labels n10, n30 and s30 denote YOLO11n trained for 10 epochs, YOLO11n for 30 epochs and YOLO11s for 30 epochs. The n30 and s30 runs form the matched architecture comparison; n10 provides context. Native validation reports mAP50 and mAP50–95 as percentages. Precision and recall use eight-class, one-to-one matching at IoU ≥0.5 on 2,623 annotated boxes.

Balanced fine-tuning produced modest gains. Combining predictions from each original image and its horizontal flip subsequently achieved box precision of 53.02% and recall of 53.26%. However, median CPU latency rose from 85.19 to 142.64 ms, and the 95th percentile rose from 99.00 to 160.94 ms. We did not deploy this combined method. The released single-v1 model uses the balanced weights and processes only the original image at 832 pixels. It selects the primary detection using the confidence-and-area score defined in Section 3.2.

Table 7. Primary image-type results on the 377-image confirmation subset

Policy	TP	FP	FN	P (%)	R (%)	Macro F1 (%)
Confidence baseline, threshold 0.25	98	62	73	61.25	57.31	42.64
Confidence-and-area selection, threshold 0.20	106	66	65	61.63	61.99	46.15

Note. These results use offline GPU exports verified by file hashes and include 171 positive images and 206 images with empty annotations. Localisation is not required for this metric. Macro F1 averages all eight classes, assigning zero to the class with no positive examples.

On the 377-image confirmation subset, the selected policy achieved image-type precision of 61.63% and recall of 61.99%, compared with 61.25% and 57.31% for confidence-only selection. It correctly classified eight more images but also produced four more false positives. When a correct detection also required localisation at IoU 0.5, precision and recall fell to 50.58% and 50.88%.

Across all 1,230 eligible images, image-type precision and recall were 65.68% and 65.23%. This larger set includes the calibration images used to choose the policy, so it cannot replace the confirmation results. Neither precision nor recall met the target of above 70%.

Both incorrect classification and missed or spurious detections contributed to the precision and recall of approximately 60%. Among 171 annotated positive images, the selected policy assigned the correct type to 106, assigned the wrong type to 31 and returned no detection for 34. It also returned predictions for 35 of the 206 images with empty annotations. These outcomes give 106 true positives, 66 false positives and 65 false negatives under the image-type metric. If evaluation considers only the presence of damage, precision rises to 79.65% and recall to 80.12% on the same images. This easier diagnostic does not establish recognition across the eight damage classes. Displaced boxes cannot explain the image-type shortfall because this metric does not require overlap.

Table 8. Class support and selected image-type performance

Source class	Support	P (%)	R (%)
longitudinal crack	250	73.25	71.20
longitudinal crack wide	25	39.39	52.00
transverse crack	74	66.67	62.16
transverse crack wide	5	75.00	60.00
alligator crack	159	75.51	69.81
alligator crack sunken	19	43.75	36.84
pothole	41	34.62	43.90
pothole deep	8	23.08	37.50

Note. These class results use all 1,230 eligible validation images, including calibration images. Support is the number of annotated positive images, rather than the number of boxes or independently captured scenes.

The class distribution and observed confusions identify areas for further investigation. The full eligible set contains 250 positive images of longitudinal cracks but only eight of deep potholes. The confirmation set contains no positive image of a wide transverse crack. In a separate box-level audit of the full validation set, 146 of 190 matched pairs with an incorrect class involved confusion between subtypes of the same damage family.

Recall was also lower for small annotated regions: 45.4% for regions covering less than 1% of image area, compared with 75.0% for those covering at least 20%. These findings support further analysis of subtypes and object scale. The audit confirmed no annotation defect.

The model still labelled a known pothole photograph as an alligator crack, with confidence of approximately 0.253, so human correction was required. The comparisons show that increasing model capacity and relaxing filtering alone do not establish reliable recognition: selection rankings and thresholds trade false positives against missed detections.

Precision among tasks accepted at the 0.60 confidence gate remains to be measured. Section 6.2 therefore prioritises independent local data and evaluation of acceptance precision, review workload and errors that escape review, while retaining all eight labels.

5.2 Routing Performance

The best-known solution (BKS) is the best result under the lexicographic ranking among 41 runs for each scenario; it is not a proven optimum. Objective gaps are compared only when the numbers of unassigned P1, P2 and P3 tasks match, while runs with inferior coverage are counted separately. Even matching priority counts can involve different tasks. A lower objective therefore does not necessarily represent travel savings for the same workload.

The Greedy-seeded GA improved on Greedy in all 120 runs under the default search settings. Across scenarios, its mean objective reductions ranged from 0.73% to 3.62%, and its mean gaps from the BKS ranged from 0.23% to 1.82%, compared with 1.17–5.10% for Greedy. Its average runtime per scenario ranged from 104.6 to 361.9 ms, while Greedy took less than one millisecond. Including the Greedy solution and retaining elite solutions prevents the GA from returning a worse result than that baseline. The size of the improvements and the comparison with pure GA are therefore more informative than the absence of regression alone.

Table 9. Greedy and pure GA on complete 20-task scenarios

Matrix / tasks / shift	Travel G / GA	Makespan G / GA	Priority completion G / GA	Objective J G / GA
Simulated / 20 / Unlimited	227.8 / 242.6	722.1 / 721.1	438.9 / 402.2	1001.0 / 983.6
Google / 20 / Unlimited	282.9 / 270.3	736.6 / 725.9	441.6 / 409.8	1030.4 / 997.7

Note. Each entry gives the Greedy value followed by the pure-GA mean over seeds 1–10. All tasks are assigned. Values are in minutes, except for the weighted objective J. A lower J does not require every component to improve, as the simulated travel results show.

Table 10. Representative mean objective gaps to the best-known solution

Matrix / shift / tasks	Greedy gap	Pure gap	Pure inferior coverage	Seeded gap
Simulated / 480 / 10	1.17	0.68	0/10	0.43

Matrix / shift / tasks	Greedy gap	Pure gap	Pure inferior coverage	Seeded gap
Simulated / 480 / 15	1.49	0.49	0/10	0.23
Simulated / 480 / 20	4.29	0.73	0/10	1.07
Simulated / Unlimited / 20	4.27	2.46	0/10	0.50
Google / 480 / 15	2.19	0.85	5/10	0.23
Google / 480 / 20	3.53	0.90	0/10	0.93
Google / Unlimited / 20	5.10	1.77	0/10	1.82

Note. Gaps are percentages averaged over ten seeds. For pure GA in the Google/480/15 scenario, the gap uses the five runs with matching coverage; the other five runs remain recorded as coverage failures. The selected rows include capacity stress cases and cases where seeded GA had a larger gap. Full records are available in SystemCode/experiments/routing/results/.

In the Google/480/15 scenario, pure-GA seeds 2, 4, 5, 7 and 10 assigned 12 tasks, compared with 13 for Greedy. Some remaining capacity was concentrated in one crew, but no additional task could be inserted directly. Assigning more work would therefore require moving tasks between crews. Greedy seeding restored 13 assignments in all ten runs. However, the seeded GA had larger mean gaps than pure GA in three 20-task scenarios. Seeding thus provides a dependable starting point without outperforming pure GA in every scenario.

The 15-task workload requires 1,920 service minutes, using the combined capacity of four 480-minute shifts before travel is included. Under the shift limit, the simulated matrix leaves three tasks unassigned, while the Google matrix allows one additional P3 task to be assigned. With 20 tasks, both matrices leave eight tasks unassigned, but all P1 tasks are assigned. Compared with Greedy, seeded GA reduces mean travel from 194.9 to 187.1 minutes and mean makespan from 477.9 to 474.6 minutes on the simulated matrix; on the Google matrix the reductions are smaller for travel (194.1 to 193.4 minutes) and makespan (477.1 to 473.0 minutes). The assigned task identities may differ between plans. Unlimited shifts allow all tasks to be assigned in the 15- and 20-task cases, but the resulting schedules exceed 480 minutes. The 10-task cases fit within the limit.

The Google experiments used historical travel estimates that did not account for traffic, while browser acceptance checks used simulation. API waiting time and algorithm runtime were recorded separately.

5.3 Integrated Workflow Verification

The integration record contains 20 acceptance groups and 201 HTTP requests sent through the frontend proxy. These checks cover uploads using the real model, human correction, invalid-image handling, idempotency, task completion, disabled crews and stale plans. In the constructed 20-task case, both planners assigned 12 tasks and reported eight exclusions due to the shift limit, without omitting any P1 task.

Table 11. Recorded release checks and workflow boundaries

Evidence or case	Recorded result	Interpretation
HTTP integration	20 groups / 201 requests	Main workflow verified through the frontend proxy
Authenticated release d48b877	973 backend passed / 1 skipped; 152 frontend; 49 CI guardrails	Evidence from regression tests and remote CI
0.60 / 0.5999; no detection	Automatic task at 0.60; review at 0.5999; no active task when no detection is returned	Controlled branch checks, not model calibration
Duplicate, stale or conflicting write	Replay or version conflict	No duplicate task created; stale plans rejected
Unavailable crew or invalid role	Explicit unassigned reason or access rejection	Tasks retained; role and crew access restrictions enforced
21 October assistant and task adjustments	Four direct-answer cases with no provider calls; two actual grounded GPT calls; separate HTTP and browser records	Targeted checks of grounding and confirmed changes that persist

Evidence or case	Recorded result	Interpretation
21 October release PR 36, merged as b8d0fdea	GitHub CI recorded 1,255 backend tests passed and 1 skipped, 179 frontend tests passed, and successful lint, type and build checks. Separately, 49 local infrastructure guardrail tests passed.	GitHub run 39569241424 checked head 155d3183. Base CI excludes optional vision dependencies and skips the YOLO test.

Table 11 summarises the checks performed at different stages of development. Some checks cover the same tests, so their counts are reported separately rather than added together. These checks assess whether the application behaves as intended, complementing the recognition and routing experiments in Sections 5.1 and 5.2. Table 12 summarises how this evaluation addresses the original project objectives.

We tested how the assistant handled both simple questions and questions requiring supporting evidence. Four direct-answer cases were handled locally without contacting the AI service. Two further cases used GPT-4.1-mini to answer questions based on project information. For the deep-pothole question, the answer correctly stated the project's P1 priority and 240-minute repair duration. For the saved-route question, it reported 98.8 minutes of travel and 165.0 minutes of service, matching the stored plan. These results apply to the knowledge base available during testing; the two GPT cases were not repeated after the subsequent knowledge-base maintenance and documentation update.

The task-adjustment acceptance record adds isolated, authenticated HTTP and browser checks covering all four route cards, move and cancellation previews, explicit confirmation and recalculation, persistent assignment constraints, and changes in crew access. Constructed routes test rejection when work exceeds the shift limit while retaining other tasks.

Table 12. Evaluation coverage against the original project objectives

Planned area	Delivered evidence and remaining gap
Image recognition	Validation experiments and the historical n10 test are recorded; the final model has no independent local test.

Planned area	Delivered evidence and remaining gap
Confidence gate	Boundary, no-detection and replay tests are recorded; accepted-case precision and review rates still require evaluation.
Priority rules	All eight labels were tested through automatic and corrected-review workflows. Frozen policies were checked, but service assumptions remain unvalidated.
Route planning	Experiments cover four crews and 10, 15 and 20 tasks. Accepted inspections now trigger an attempt to generate paired plans automatically; other changes require explicit replanning. Lateness against deadlines has not been evaluated.
GPT/RAG	Targeted evidence covers grounded repair and saved-plan answers, direct intent routing and confirmed adjustments through actual-provider calls, local retrieval tests, and controlled HTTP and browser checks. Completion of the full proposed ten-question and ten-change suite is not established. Broad evaluation of answer meaning and user experience remains outstanding.
Whole system	Dated desktop and HTTP acceptance records and separate mobile checks are available; user productivity has not been measured.

6 Conclusions and Future Work

6.1 Conclusions and Limitations

RoadGuard connects image-based damage detection to repair decisions and crew schedules in one application. Inspectors submit photographs, dispatchers review uncertain results and adjust work, and crews view their routes and record completion. Stored evidence, citations and revision checks help explain how each decision was made. The experiments show modest recognition gains and better scheduling objectives with Greedy-seeded search. They also reveal unresolved classification errors, a pure-GA assignment failure and gaps in assistant evaluation.

Generalisation is limited by reused validation data, a single training seed, sparse damage subtypes, one task family and one Google travel snapshot. Service durations, shift and return policies, and objective weights are project assumptions requiring operational validation. Routing results compare constructed scenarios against an empirical best-known solution, and statistical significance has not been established. Independent local vision testing, broader assistant and user evaluation, and measurements of operational savings remain future work.

6.2 Future Work

Future vision work should review subtype labels, missed potholes and disputed negative examples, retaining corrections only after verification. The dataset should include more examples of rare classes and hard negatives such as patches, wet surfaces, shadows, gravel and manholes. Local collection should include phone photographs taken from a moderate distance, with road context, annotations for all objects and an identified primary subject.

An independently labelled local acceptance set should be frozen, with images grouped by location, defect and adjacent frames. Comparisons of sampling strategies, context-preserving augmentation and class calibration should use only training and calibration data. Model size and resolution should be varied one at a time, with repeated seeds, latency checks and checkpoints that support recovery. Evaluation should report eight-class precision and recall, misses by class, localisation, precision among automatically accepted cases and the review rate separately.

Future routing evaluation should use verified access information, measured service durations, travel estimates from varied periods and independent task sets. Comparisons with bounded or exact solutions on small instances would help assess solution quality. Coordinated moves between crews could address cases where direct insertion fails. A repair step that relocates a task and then inserts an unassigned one, or local search such as 2-opt and inter-crew relocation applied to elite solutions, would follow the hybrid approach discussed in Section 1.2. A representation that stores each crew's sequence directly would also remove redundant solutions in which only the relative order of different crews' tasks differs. The assistant and confirmed-adjustment workflows also need broader evaluation of answer quality, citations, multilingual ambiguity and recovery, systematically covering the original question and change cases.

Appendix A Installation and User Guide

This guide covers local installation, the workflow from inspection to completion, and the dispatcher assistant. Local acceptance checks used Windows, Python 3.12.14 and Node 22.23.3. Installation on an independent clean machine has not been validated.

A.1 Environment and dependencies

Use Python 3.12 for the full application because the pinned numpy==2.5.2 requires Python >=3.12. Base CI uses Python 3.11 and excludes the optional vision dependencies, so its success does not establish that vision runs on Python 3.11. Use Node 22.x >=22.12 and npm >=10.

Clone the public GitHub repository. The approved YOLO checkpoint is included in SystemCode/backend/roadguard/model/, and the Dataset directory is not required for inference. On first use, create a new environment from the repository root:

```
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r SystemCode/backend/requirements-ci.txt
.\.venv\Scripts\python.exe -m pip install -r SystemCode/model/requirements-vision.txt
```

If an existing .venv uses another Python version, create a separate Python 3.12 environment and use its path in every command below. Do not recreate an environment while it is in use. The recorded local acceptance tests support CPU inference. GPU training has separate CUDA requirements.

A.2 Local retrieval and optional GPT configuration

Install the full backend requirements and prepare the reviewed local knowledge index before using evidence questions. Run these commands from the repository root after creating the Python 3.12 environment:

```
.\.venv\Scripts\python.exe -m pip install -r SystemCode/backend/requirements.txt
$env:PYTHONPATH = 'SystemCode/backend'
.\.venv\Scripts\python.exe -m roadguard.assistant.index_knowledge --download-model
```

On first setup, the command downloads approximately 487 MB of pinned multilingual-e5-small model and tokenizer files and verifies their hashes. The encoder runs on CPU through

ONNX Runtime. It does not call an embedding API or change the YOLO model. Later setup runs reuse the verified files.

After pulling a reviewed knowledge update, rebuild the current index with the following command:

```
.\.venv\Scripts\python.exe -m roadguard.assistant.index_knowledge
```

Then restart the backend. HTTP requests never download models or rebuild indexes. The current knowledge package is a-knowledge-v19. Chroma collections are versioned against the manifest, encoder and chunking policy.

The default Chroma path is SystemCode/backend/data/rag/chroma/, and embedding files are stored under SystemCode/backend/data/rag/models/multilingual-e5-small/. Both locations hold local runtime files excluded from Git. To override them on the server, use ROADGUARD_CHROMA_PATH and ROADGUARD_EMBEDDING_PATH.

The index excludes the dataset, uploads, account records and business databases. Retrieval uses cosine similarity, which measures similarity rather than answer correctness. See the vector retrieval guide (../SystemCode/backend/roadguard/assistant/VECTOR_RETRIEVAL.md).

GPT generation is disabled by default. To enable it, set these variables privately in the backend terminal before startup:

```
$env:ROADGUARD_ASSISTANT_ENABLED = 'true'
$env:ROADGUARD_ASSISTANT_MODEL = 'gpt-4.1-mini'
$privateKey = Read-Host 'OpenAI API key' -AsSecureString
$env:OPENAI_API_KEY = [System.Net.NetworkCredential]::new('', $privateKey).Password
Remove-Variable privateKey
```

The recorded targeted acceptance tests used gpt-4.1-mini. Select a model available to your account. The enabled flag accepts exactly lowercase true or false; uppercase True is invalid. Restart the backend after changing environment settings. Keep the key out of frontend code, Git and the report. The status endpoint reports non-secret configuration and usage, but does not verify credential validity.

The optional limits are ROADGUARD_ASSISTANT_MAX_OUTPUT_TOKENS (default 1200, range 256–2000) and ROADGUARD_ASSISTANT_MAX_CALLS (default 20 per process, range 1–200). Every attempted call, including a failed call, consumes the local budget. This budget resets on restart and does not cap account-wide spending. Real provider requests

may incur charges. The assistant does not automatically retry requests or fall back to another model.

A.3 Accounts and data

No default accounts or shared passwords are shipped. Provision accounts in the same database used by the server; each command privately prompts for a 12–256-character password:

```
$env:PYTHONPATH = 'SystemCode/backend'
.\.venv\Scripts\python.exe -m roadguard.auth.manage create inspector --role inspector
.\.venv\Scripts\python.exe -m roadguard.auth.manage create dispatcher --role dispatcher
.\.venv\Scripts\python.exe -m roadguard.auth.manage create crew02 --role crew --crew crew-02
```

When needed, provision distinct usernames for crew-01, crew-03 and crew-04. Each account has a fixed crew identity. Accounts and passwords are stored in the local database and are not shared through Git.

The default database is SystemCode/backend/data/roadguard.sqlite3. Back it up together with the adjacent uploads directory. If you customise ROADGUARD_DB_PATH, use the same value when provisioning accounts and starting the server. See account roles (../Miscellaneous/ACCOUNT_ROLES.md).

A.4 Start the application

Backend, from the repository root:

```
$env:ROADGUARD_TRAVEL_MATRIX_PROVIDER = 'simulated'
.\.venv\Scripts\python.exe -m uvicorn roadguard.main:app --app-dir SystemCode/backend --host 127.0.0.1 --port 8000
```

In a second terminal, from the repository root:

```
cd SystemCode/frontend
npm ci
$env:ROADGUARD_API_TARGET = 'http://127.0.0.1:8000'
npm run dev
```

Open http://127.0.0.1:5173 and sign in. API documentation is available at http://127.0.0.1:8000/docs, and the public health endpoint is /api/v1/health. The health check verifies application and database access only. The model loads when real inference is first requested.

To stop a server, press Ctrl+C in its terminal. Do not start duplicate servers on occupied ports.

A.5 User workflow

1. Sign in as an inspector. Upload a JPEG or PNG within the limits of 8 MiB and 20 megapixels. Choose real inference or an explicit synthetic scenario, enter Jurong coordinates and declare the road type. Example coordinates such as 1.3405, 103.706 are synthetic unless verified as the photo's location.

2. Inspect the result. The upload response contains inspection_id, status, one detection or null, and task_id. For real inference, a primary detection with raw confidence >=0.60 creates one task. A lower confidence requires review, while no detection creates no task. Open the inspection details to view the raw evidence and the frozen rules used for the decision.

3. Sign in as the dispatcher. Confirm, correct or reject pending inspections. Corrections use the stored rules, and accepted tasks contain the demonstration priority and service duration. Completed and cancelled records retain their history but are excluded from the active planning snapshot.

4. An accepted upload or review confirmation/correction automatically starts a paired planning attempt after the task is committed. GA and Greedy use the same task snapshot and travel matrix, and both honour crew constraints. Inspect any unassigned-task reasons and compare the results from the same planning round. If the browser no longer retains a matching response, use the manual recalculation button to obtain a comparison from one round. Manual recalculation is also required after other input changes or a failed automatic attempt.

The upload or review response waits for the planning attempt. X-RoadGuard-Replanning reports completed, failed or not_needed, while the four upload JSON fields remain unchanged. If planning fails, the task is preserved, the previous plans remain stale and a warning appears. Repeating the accepted request recovers missing or stale plans and skips replanning when the pair is already current.

Automatic planning is serialized within each server process. It allows at most three attempts in total, or two retries, and retries only stale conflicts. There is no durable worker to resume

retries after a restart. If the server crashes after the task is committed, repeat the accepted request or recalculate manually.

5. Sign in as the crew assigned by the latest saved GA plan. View and complete that crew's tasks. The server checks the current assignment and task version before accepting completion. Completion changes the planning inputs, so the dispatcher must explicitly replan.

The frontend map shows a schematic stop order rather than turn-by-turn navigation. See the frontend guide (../SystemCode/frontend/README.md).

A.6 Dispatcher evidence questions

Sign in as a dispatcher and open Route assistant. Ask about a supported repair rule or project operation. For a route question, select an explicit saved plan. Saved-plan answers use authorised facts stored on the server rather than totals supplied by the browser.

Questions may use English or the supported, limited set of Chinese aliases. Generated answers and the interface are in English.

Read the answer first. Sources are closed by default. Expand View sources to inspect exact quotations, citation IDs, package/version hashes and the provenance of saved-plan facts.

The query-v3 grounded_summary mode retrieves evidence locally through Chroma/E5 and may make one GPT call. GPT selects evidence IDs owned by the server, which then restores the exact quotations and required context. For canonical repair questions, the server renders answers from matching rule rows and retains the demonstration qualifiers.

Checks on numbers, priorities, identifiers, labels, sources and plans reject unsupported output. These bounded checks do not establish semantic correctness for every possible answer.

A standalone question such as “What model are you?” returns the configured model name through server_configuration. It does not call GPT, retrieve knowledge or read saved plans, and remains available when generation is disabled or the key is missing. The answer identifies the configured assistant model. It does not prove provider execution or identify the separate YOLO detector.

Clearly unrelated questions return scope_guidance with RoadGuard examples. These direct modes provide no citations or knowledge-package metadata and consume no model-call budget.

Ambiguous questions and questions that mix domain topics still follow the evidence flow. Insufficient evidence and validation failures are reported explicitly.

Queries do not change tasks, crew availability or routes. A model answer never authorizes a business action.

A.7 Dispatcher route previews and confirmed adjustments

Open Crew adjustments in Route assistant and load a saved plan ID or the latest GA plan. The four crew cards show stop order, travel, service, required return travel, total work, remaining shift and persistent assignment badges. Use a current plan when making changes. Changes to crew availability have their own preview and confirmation workflow.

To move a task, select an assigned active task and a different available crew, then preview the move. Compare the routes before and after the change, acknowledge the preview and confirm.

The server retains all other assigned work. It checks that travel, service and the configured depot return fit within the shift limit, which is 480 minutes in the routing demo. It rejects the move without changing records if travel is unknown, the work is infeasible, inputs or configuration are stale, or the preview has expired. A rejected move never silently drops another task.

Confirmation saves a durable task-to-crew constraint. To generate a new saved route, choose Recalculate saved task change and reload the latest GA plan.

Both Greedy and GA retain the constraint after a restart and when new tasks are added, although stop order may change. If the constrained crew is unavailable or the assignment is infeasible, the task is reported as unassigned rather than silently moved to another crew. The shift and return policies remain configurable demonstration assumptions.

To cancel work, choose Cancel active task, preview the cancellation and explicitly confirm it. Both assigned and unassigned active tasks can be cancelled. Cancellation preserves the task and its history but removes it from active planning. Explicitly recalculate to replace stale routes. Completed and cancelled tasks cannot be reopened.

Discard task preview closes only an unconfirmed proposal; it does not cancel a task. After an interruption, use recent records or the change ID to recover the operation. Refresh before retrying a version conflict. Confirmation and replanning retries are bounded and idempotent.

Reassignment immediately removes the previous crew's access to read the task and inspection or complete the task. The target crew gains access through an explicitly recalculated saved GA plan. An affected old route is omitted, with an explanation that it is stale and requires recalculation.

Crew and inspector accounts cannot perform dispatcher adjustments. The backend enforces these checks as well as the interface.

A.8 API testing and troubleshooting

In Swagger, call POST /api/v1/auth/login with a provisioned account. Use the same host name throughout so that session cookies are retained. For authenticated requests that modify data, send the returned csrf_token as X-CSRF-Token.

An upload uses multipart image and metadata fields. Use a new Idempotency-Key for each new submission. If the response is lost, retry with identical content and the same key.

Table A1. Common issues

Issue	Action
Dependency installation rejects NumPy	Check that the environment uses Python 3.12. Base CI uses Python 3.11 and does not include vision dependencies.
Connection refused	Start the backend and check that the frontend proxy target matches its address.
401/403	Check sign-in, account role, session host and CSRF token.
Database locked	End outstanding viewer transactions. Do not delete the database.
Real inference unavailable	Install the vision requirements and verify the bundled weight and manifest. Restart after a failed model load.
Stale plan or 409	Refresh the current state, then explicitly replan or retry with the current record version.
Unassigned tasks	Read the reported reasons, such as unavailable crews or shift limits.

Issue	Action
Embedding model unavailable or index missing/stale	Install the full backend requirements, verify the configured paths, run the explicit index command and restart. HTTP requests do not repair indexes.
Invalid assistant configuration	Use lowercase true/false, a valid model name and the documented limit ranges, then restart.
GPT unavailable, quota or provider error	Check private server configuration, account access and the local attempted-call budget. Model identity remains available. Do not repeatedly retry requests whose outcomes are unknown.
Insufficient evidence or rejected answer	Narrow the question or select a current saved plan. Inspect the explicit error without bypassing citation validation.
Task preview expired or 409	Refresh the task, plan and configuration, then create a new preview before confirming.
Move exceeds shift or travel is unknown	Inspect travel, service and return totals and crew availability, then choose a feasible crew. Other tasks are retained.
Constrained task unassigned	Inspect the locked crew and the reported reason. Later recalculation preserves the constraint rather than silently transferring the work.
Crew access lost after reassignment	The dispatcher must explicitly recalculate. The target crew can then refresh its latest saved GA route.
Google provider error	Check server credentials and service access. Provider errors do not silently switch the system to simulation.

A.9 Optional Google travel estimates

Set these variables in the backend terminal, then restart and explicitly recalculate:

```
$env:ROADGUARD_TRAVEL_MATRIX_PROVIDER = 'google_routes'
$env:ROADGUARD_GOOGLE_MAPS_API_KEY = '<your private key>'
```

Usage may incur charges. Keep the key private and out of frontend source. The current provider uses traffic-unaware estimates, and the map remains schematic. Existing plans retain their original travel source.

The recorded browser acceptance tests used simulation and did not make a new real-Google call. See the routing guide (../SystemCode/backend/roadguard/routing/README.md).

Additional operating details: the pinned vision requirements include PyTorch, torchvision and Ultralytics. The example crew02 account can act only for Crew 02 and may have no assigned work. For the crew workflow, use the crew assigned in the latest saved GA plan.

After an upload, open the detail page to inspect provenance and stored rule information. A changed upload payload requires a new submission and Idempotency-Key. GET requests do not require CSRF.

Example real-upload metadata, supplied with the multipart image:

```
{"mode":"real","latitude":1.3405,"longitude":103.706,
 "road_type":"public_motor_road"}
```

Appendix B Submitted Project Proposal

The submitted proposal is reproduced in full, including its original contents, diagrams and pagination. Its cover retains the former Group 07 identifier; the current group is 37. The proposed two-crew arrangement records the original plan. Sections 1.3 and 5.3 explain subsequent changes to scope and evaluation. Report page numbering continues through the appendix.

Submitted proposal - source page 1 of 13

Intelligent Reasoning Systems

Project Proposal

Project Name: RoadGuard

Group 07

YanGuanting A0305485B

HuErji A0305480L

WuTaoyi A0356972M

Submitted proposal - source page 2 of 13

Contents

1. Executive Summary 1
2. Background and Problem Statement 1
    2.1 Real-world need 1
3. Business Value and Users 2
4. Objectives and Scope 3
    4.1 Objectives 3
    4.2 MVP scope 3
5. Data Plan 4
6. System Design 5
    6.1 Workflow 5
    6.2 Simplified architecture 6
7. Functional Requirements 7
    7.1 Front-end 7
    7.2 Back-end 7
8. Reasoning and Route Planning 8
    8.1 Core rules 8
    8.2 Route optimization 8
    8.3 GPT route assistant and RAG 9
9. Implementation and External APIs 9
10. Evaluation Plan 10
11. Risks, Deliverables and Conclusion 10
    11.1 Main risks 10
    11.2 Deliverables 11
    11.3 Conclusion 11

Submitted proposal - source page 3 of 13

1. Executive Summary

RoadGuard is a web-based system that turns a road photo and location into a maintenance task and updated repair routes. An inspector uploads a road image and its location, then YOLO predicts the damage type, severity and confidence. If the confidence meets a configurable threshold, preset rules will automatically set the priority, create the task, add it to the queue and trigger route recalculation. Only low-confidence results wait for human review. A group-built Genetic Algorithm (GA) will use Google travel times to plan routes for two crews, while GPT with RAG system will explain the route and help users request a confirmed route change.

2. Background and Problem Statement

2.1 Real-world need

Road inspection and maintenance involve repeated observation, judgement and coordination. In practice, staff may receive many road images, but manually checking every clear result slows down task creation. Damage severity may also be judged differently by different people, and a new urgent task can quickly make an existing crew route outdated. RoadGuard connects image recognition, preset decision rules, task queues and route recalculation in one workflow. Human attention is kept for low-confidence cases, while clear cases can move forward automatically.

The project focus on five practical gaps:

Operational problem	**Effect**	**RoadGuard response**
Image review workload	Many images need the same repetitive checking	YOLO predicts damage type, severity and confidence
Uncertain predictions	Wrong detections may create wrong tasks	A confidence gate sends only low-confidence cases to human review

Submitted proposal - source page 4 of 13

Operational problem	**Effect**	**RoadGuard response**
Different priority decisions	Similar cases may be treated differently	Preset rules apply the same priority mapping
New tasks make routes outdated	Crews may follow an old route plan	Queue changes automatically trigger route recalculation
Routes are hard to explain or change	Maintenance coordinator may not trust or use the route plan	GPT with RAG explains the route and converts confirmed requests into inputs for a new route calculation

3. Business Value and Users

User	**Main need**
Inspector / reporter	Upload an image and location
Maintenance coordinator	View the automatic queue and updated routes, ask for an explanation or request a route change,review only a low-confidence result when requested
Crew	View the maintenance queue and route map

Expected value is expressed as measurable project outcomes rather than promised financial savings:

- Clear image results can become tasks without waiting for routine manual review.
- Preset rules make severity-to-priority decisions consistent and easy to check.
- Every accepted new task updates the queue and triggers a new route calculation.
- Route planning considers urgency, travel time and crew workload.
- GPT gives a short explanation supported by the project knowledge base and handles only confirmed change requests

Submitted proposal - source page 5 of 13

4. Objectives and Scope

4.1 Objectives

1. Detect the eight RDD2022ES labels that combine damage type with a basic severity distinction.
2. Use a configurable confidence threshold to separate automatic processing from human review.
3. Use preset rules to set P1/P2/P3 priority and repair advice.
4. Automatically create an accepted task, update the queue and recalculate two crew routes.
5. Use Genetic Algorithm to assign and order tasks for two crews.
6. Use GPT API with RAG system to explain the chosen route and process a user-confirmed route change.

4.2 MVP scope

In scope	**Out of scope**
Single-image upload or phone camera input	Continuous video or dashcam processing
Eight RDD2022ES damage/severity labels	Engineering-grade depth, width or structural measurement
Automatic flow above the threshold; Human review below the threshold	Unsupervised real-world deployment without logs or override
Preset priority rules and automatic queue update	Real staff, stock or contractor systems
Two-crew GA and automatic recalculation	City-wide live dispatch
GPT route explanation and a few confirmed change types	GPT directly creating an unchecked route
Responsive React web app	Native iOS or Android app

Submitted proposal - source page 6 of 13

5. Data Plan

RDD2022ES is the main image dataset. It is a manually relabeled severity extension of RDD2022 with about 25,000 images, including mirrored copies, and eight detection labels: longitudinal crack / wide, transverse crack / wide, alligator crack / sunken, and pothole / deep. These labels make a basic severity-aware model easier to train than plain RDD2022, but they do not provide a complete engineering condition score. The RAG system uses a few reviewed project files containing priority rules, repair notes, route assumptions and a user guide.

Data type	**Plan**
Image model	RDD2022ES boxes with eight damage/severity-aware labels; balanced train, validation and test splits
Route test	10/15/20+ made-up tasks, one fictional department and two crews
RAG knowledge	A few reviewed files about preset rules, repair advice, route assumptions and user guide

Submitted proposal - source page 7 of 13

6. System Design

6.1 Workflow

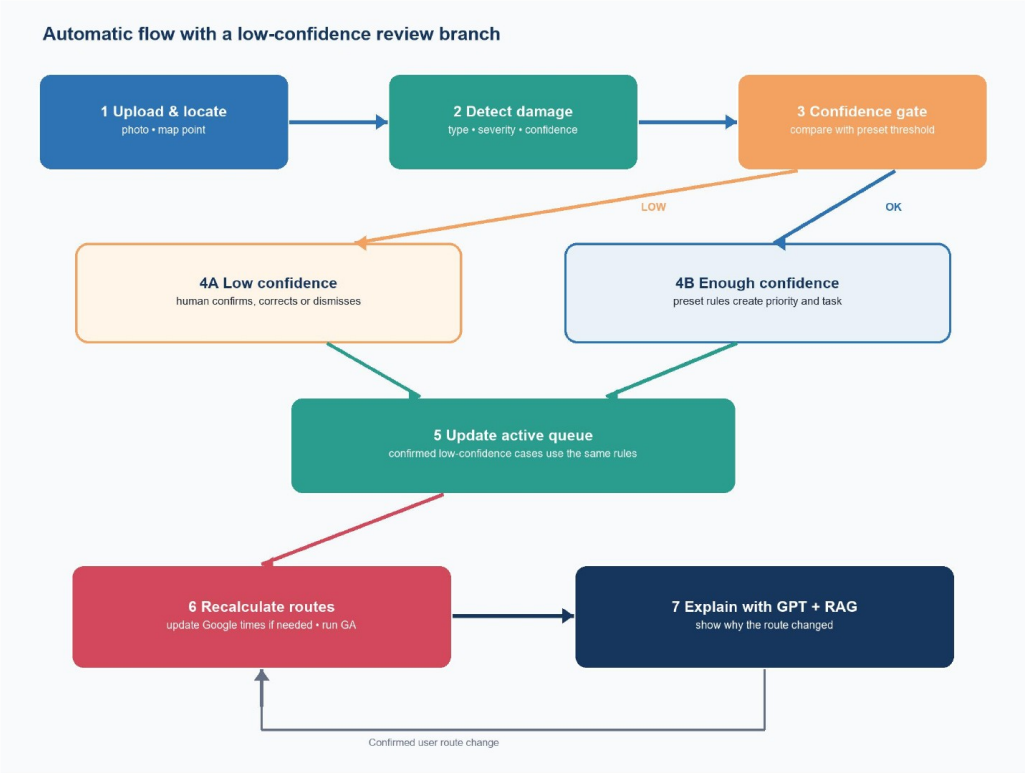


Figure 1. RoadGuard workflow

The confidence threshold is previously set and can be tuned on validation data. When confidence is high enough, the system applies the preset rules, creates the task and adds it to the active queue automatically. When confidence is too low, a staff (coordinator) confirms, corrects or dismisses the result; a confirmed result then follows the same preset rules. Every active-queue change updates any required Google travel times and triggers the GA again. GPT does not calculate the route, it explains the new result using route facts and RAG knowledge. A user-requested route change is still shown for confirmation before recalculation.

Submitted proposal - source page 8 of 13

6.2 Simplified architecture

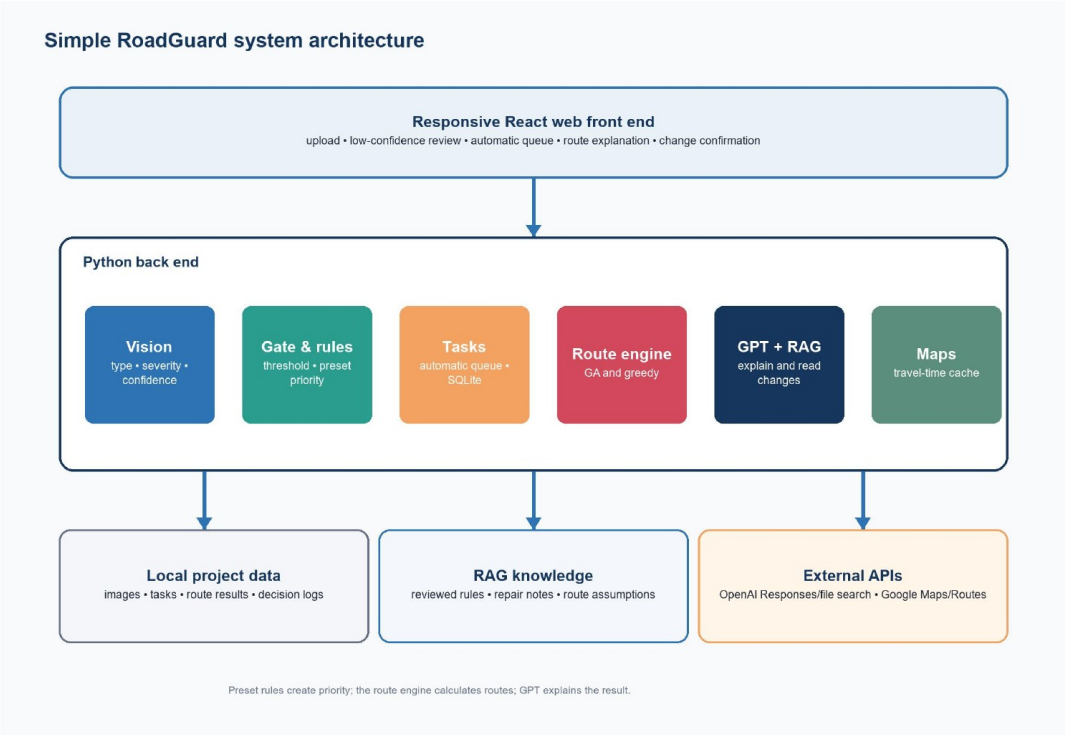


Figure 2. Simple web system

The whole system uses a React web front-end and a Python back-end. YOLO returns damage type, basic severity and confidence. A confidence gate and preset rules decide whether a case can enter the task queue automatically or must wait for human review. A queue change calls the route engine, which uses cached Google road travel times. The GPT with RAG knowledge base explain the calculated result and read supported change requests.

Submitted proposal - source page 9 of 13

7. Functional Requirements

7.1 Front-end

Page	Essential functions
Inspection	Upload or take a photo, choose a location, and view damage type, severity and confidence
Review queue	Show only low-confidence cases; confirm, correct or dismiss them; priority still comes from preset rules
Tasks and routes	Show the automatically updated queue, two recalculated crew routes and route metrics
Route assistant	Ask why the route planning is like this; enter a supported change request and confirm it before route recalculation
Evaluation	View image, confidence-gate, rule, route and GPT/RAG test results

7.2 Back-end

Module	Responsibility
Vision	Run YOLO and return marked boxes, damage type, basic severity and confidence
Confidence and rules	Send low-confidence cases to human review; otherwise apply preset priority rules
Tasks	Create accepted tasks, update the active queue and save the decision trace in SQLite
Route engine	Recalculate greedy and GA routes whenever the active queue changes

Submitted proposal - source page 10 of 13

Module	Responsibility
GPT and RAG	Find relevant notes, explain updated routes and read supported change requests
Maps	Get new Google travel times when needed, cache them and provide an offline backup

8. Reasoning and Route Planning

8.1 Core rules

Rule	Condition	Result
R01	Confidence < 0.60	Wait for human review; do not add an active task
R02	Confidence ≥ 0.60	Apply preset rules, create task and update the queue
R03	Wide, sunken or deep severity label	High severity; P1
R04	Base pothole or alligator-crack label	Medium severity; P2
R05	Base longitudinal or transverse crack	Low severity; P3

8.2 Route optimization

The GA creates two ordered task lists, one for each crew. It tries different task orders and keeps routes that reduce travel time, late urgent work and unfair workload. A simpler greedy method is used as a comparison. When a task enters or leaves the active queue, the route service updates the saved Google travel-time matrix for any new location and runs both methods again. Severity helps create the priority input but it will not replace the route calculation.

Submitted proposal - source page 11 of 13

8.3 GPT route assistant and RAG

After each route calculation, RAG system finds useful priority rules, repair notes and route assumptions from a small reviewed knowledge base. GPT receives those notes together with the selected route, travel time, priorities and crew workload, then generates a short explanation of why routes were planned like this. It also reads a limited set of route-change requests, such as an unavailable crew, a blocked road or a new urgent task. GPT itself will never change priority or the stored route.

9. Implementation and External APIs

Area	**Choice**
Front-end	React
Image recognition	Ultralytics YOLO nano or small
Rules	Python rule engine and rules.yaml
Route planning	Python GA and simple greedy comparison
Storage	SQLite and local image folders
Google	Maps JavaScript API and Routes API route matrix
GPT	OpenAI Responses API
RAG	ChromaDB with a few approved documents

Submitted proposal - source page 12 of 13

10. Evaluation Plan

Area	Experiment	Metrics
Image model	Held-out RDD2022 images	Per-class Precision, Recall, F1 and mAP@0.5
Confidence gate	Automatic and review cases at several thresholds	Auto-accept precision, review rate and missed-error examples
Priority rules	20–30 normal and boundary cases	Expected result, actual result and pass rate
Route planning	Add tasks to 10/15/20-task queues; GA versus greedy	Travel time, late tasks, workload balance, runtime and successful recalculation
GPT and RAG	10 route questions and 10 supported change requests	Correct source, clear explanation, correctly read change and safe refusal
Whole system	Five desktop or phone-browser scenarios	Correct confidence branch, queue update, route update and error handling

11. Risks, Deliverables and Conclusion

11.1 Main risks

Risk	Mitigation
RDD2022ES dataset does not match Singapore roads	Use a small local test, report the gap and keep low-confidence review
Confidence threshold accepts wrong results	Tune it on validation data, log automatic decisions and allow later correction

Submitted proposal - source page 13 of 13

Risk	Mitigation
GPT gives an unsupported explanation	Use reviewed RAG files, fixed output fields and confirmation for changes
Google/OpenAI cost, network or privacy issue	Cache data, limit API use, minimize shared fields and keep a demo fallback

11.2 Deliverables

- Runnable React web app with a user guide.
- RDD2022ES YOLO model, confidence gate, preset rules, automatic queue, GA, greedy method, GPT prompts and RAG files.
- Test results for severity-aware detection, confidence routing, priority rules, automatic route updates and GPT/RAG behaviour.

11.3 Conclusion

RoadGuard keeps one clear workflow: detect damage type, severity and confidence; process reliable results automatically; human review only low-confidence results; apply preset priority rules; update the task queue; recalculate routes and explain the result. RDD2022ES dataset makes the severity-to-priority step more direct, while Google travel times and the group-built GA still perform the actual route calculation. GPT with RAG system makes the result easier to understand and adjust.

Appendix C Course Functionality Mapping

Table C1 maps the implemented functions to the MR, RS and CGS course content and the three applicable technique groups in the practice briefing. It points to the design and evaluation evidence in the main report without repeating the experimental results.

Table C1. Course content applied in the delivered system

Course and technique	**Applied function**	**Implementation evidence**	**Verification evidence**
MR Decision automation	Confidence routing, repair rules and human correction create tasks. Separately confirmed adjustments govern reassignment and cancellation.	Versioned policies, stored rule snapshots and decision traces. Preview validation and durable crew constraints support adjustments. Accepted inspections trigger automatic planning; other changes require explicit replanning.	Sections 2, 3.2 and 5.3 cover boundary, correction, replay and controlled adjustment checks.
RS Evolutionary resource optimisation	Assign tasks and order visits for four crews. Priority-first coverage takes precedence over the weighted completion, travel and workload objective.	The group-built GA uses encoding, selection, crossover, mutation and elitism. GA and Greedy share the evaluator, task snapshot and travel matrix.	Sections 3.2, 4 and 5.2 cover 10/15/20-task scenarios, multiple seeds, a seeding ablation and analysis of unassigned-task failures.

Course and technique	Applied function	Implementation evidence	Verification evidence
CGS Learned perception and grounded explanation	YOLO11s predicts eight damage classes and bounding boxes. Primary selection supplies evidence to the rules. Retrieval and optional GPT explain reviewed knowledge and authorised saved plans.	Grouped data, supervised training and local inference support perception. A static Chroma/E5 index, query-v3 exact citations and server-derived rule answers support explanation. Plan facts remain outside the index.	Sections 2–5 report recognition metrics and a failure audit, with separate actual-GPT, local-retrieval and controlled-case evaluations.

Storage, authorisation and interface display support these three technique groups.